

A Lattice Solution to Approximate Common Divisors

Yung-Hsiang Liu, Li-Ting Tsai, Chia-Wen Hsieh, and Rong-Jaye Chen

Department of Computer Science

National Chiao-Tung University, HsinChu, Taiwan

{liuyh,ldtsai,cwhsieh,rjchen}@cs.nctu.edu.tw

Abstract

The approximate common divisor problem (ACDP) is to find one or more divisors which is the greatest common divisor of the approximate numbers a and b of two given numbers a_0 and b_0 . Howgrave-Graham[7] has considered the special case of $b = b_0$ and gave a continued fraction approach and a lattice approach to find divisors. Furthermore he raised another lattice approach for ACDP based on Coppersmith's method[3]. In this paper, we first propose a strategy to efficiently generate two bivariate independent polynomials via LLL reduction method. Then we use bivariate Newton's method to find the roots of these two polynomials and thereby we find the divisors.

Keywords: Approximate common divisors, lattices, LLL, Coppersmith's method, Newton's method, fully homomorphic encryption

1 Introduction

This approximate common divisor problem is first introduced in 2001 by Howgrave-Graham. Given inputs a_0 , b_0 , and bounds X , Y and D , the problem asks for all d 's such that $d \mid a_0 + x_0$ and $d \mid b_0 + y_0$ for some $d > D$, $|x_0| < X$ and $|y_0| < Y$. If $Y = 0$, then b_0 is exactly known. Therefore we have a partially approximate common divisor problem (PACDP). If neither input is exactly known, then it is called a general approximate common divisor problem (ACDP).

Howgrave-Graham also gave two algorithms to solve the ACDP. One is a continued fraction approach, and the other is a lattice approach based on Coppersmith's method. This paper develops a strategy applied to the lattice approach, and provides the implementation details and a toy example for clear explanation.

A fully homomorphic encryption scheme is also raised based on the hardness of the ACDP[4]. It brings out the importance of the ACDP. In section

2, the fully homomorphic encryption is mentioned. The lattice solution to ACDP is described in detail in Section 3. Section 4 presents our strategy regarding the lattice solution, and computational experiments. Finally, a conclusion is given in the last section.

2 Fully homomorphic encryptions

The privacy homomorphisms are encryption functions which permit encrypted data to be operated on without decrypting it. This concept was introduced by Rivest, Adleman, and Dertouzos in 1978[9]. The problem of constructing such encryption functions is first solved by Gentry in 2009[5]. The encryption function of privacy homomorphism is nowadays called a fully homomorphic encryption.

A homomorphic public key encryption scheme $\mathcal{E}$ contains four algorithms, including $\text{KeyGen}_{\mathcal{E}}$, $\text{Encrypt}_{\mathcal{E}}$, $\text{Decrypt}_{\mathcal{E}}$, and $\text{Evaluate}_{\mathcal{E}}$. The public and private key pair of this scheme is (pk, sk) generated by $\text{KeyGen}_{\mathcal{E}}$. For a message m_i , $c_i = \text{Encrypt}_{\mathcal{E}}(pk, m_i)$ is the ciphertext. Also, the message $m_i = \text{Decrypt}_{\mathcal{E}}(sk, c_i)$. The $\text{Evaluate}_{\mathcal{E}}$ takes the public key pk , a circuit C from a permitted set C_E , and a tuple of ciphertexts $(c_1, c_2, \dots, c_t)$ as input, and outputs a ciphertext φ , where $C(m_1, m_2, \dots, m_t) = \text{Decrypt}_{\mathcal{E}}(sk, \varphi)$. A scheme is fully homomorphic if its $\text{Evaluate}_{\mathcal{E}}$ permits all operations on encrypted data.

The first fully homomorphic encryption scheme is constructed using ideal lattices. Later, another scheme was introduced by Dijk et al. using only elementary modular arithmetic. The security of this scheme is reduced to finding approximate integer common divisors. In this scheme, the four parameters, ρ , η , γ , and τ , are used. They are the bit-length of the noise, that of the secret key, that of the integers in the public key, and the number of integers in the public key, respectively, and relates to the security parameter λ .

The following describes the fully homomorphic encryption scheme $\mathcal{E}$ over the integers:

1. $\text{KeyGen}_{\mathcal{E}}(\lambda)$. The secret key is an η -bit odd integer p . For the public key, it chooses integers x_i randomly, where $x_i = pq_i + r_i$ for some integers $r_i \in (-2^\rho, 2^\rho)$, and $q_i \in [0, 2^\gamma/p]$. Then, the largest integer is named x_0 , which must satisfy $(x_0 \bmod p) \bmod 2 = 0$ and be odd. That is, q_0 is odd and r_0 is even. The public key consists of a set of integers $\{x_0, x_1, \dots, x_\tau\}$.
2. $\text{Encrypt}_{\mathcal{E}}(pk, m \in \{0, 1\})$. This outputs the ciphertext $c = (m + 2r + 2\sum_{i \in S} x_i) \bmod x_0$, where S is a subset of $\{1, 2, \dots, \tau\}$ and $r \in (-2^\rho, 2^\rho)$ is an integer.
3. $\text{Evaluate}_{\mathcal{E}}(pk, C, c_1, c_2, \dots, c_t)$. Given the binary circuit C with t inputs, and t ciphertexts c_i , it applies the integer addition and multiplication to the ciphertexts over the integers, and returns the resulting integer. The addition and multiplication on ciphertexts are the same operation as XOR and AND logical gate operation on plaintexts respectively.
4. $\text{Decrypt}_{\mathcal{E}}(sk, c)$. It outputs the plaintext $m = (c \bmod p) \bmod 2$.

Example 1. Let $p = 4067$ be the private key, and $\{x_0 = 1020807, x_1 = 760542, x_2 = 955735, x_3 = 854075\}$ be the public key. For $m_1 = 1, m_2 = 0$, by choosing $S_1 = \{1, 2\}, r_1 = 7, S_2 = \{1, 3\}, r_2 = -4$, the ciphertexts are $c_1 = 370148$ and $c_2 = 166805$. The homomorphic properties are as follows:

- The operation $(m_1 \text{ XOR } m_2)$ is computed as $c_1 + c_2 = 536953$. The decryption result is $(536953 \bmod 4067) \bmod 2 = 1$.
- The operation $(m_1 \text{ AND } m_2)$ is computed as $c_1 c_2 = 61742537140$. The decryption result is $(61742537140 \bmod 4067) \bmod 2 = 0$.

The public key in this scheme are integers, which are approximations of multiples of p , the private key. The integers of its public key form an ACDP, and the solution is the private key.

3 A lattice solution to ACDP

The lattice approach to solve ACDP uses the concept of Coppersmith's method, which employs

lattice basis reduction. Coppersmith's method derives two bivariate polynomials. Then, we apply Newton's method to find out the roots. At last, the greatest common divisor appears. The basic of the algorithm is described briefly in this section.

3.1 Lattice basis reduction algorithm

Let $b_1, \dots, b_d \in \mathbb{Z}^n$ be a set of linearly independent vectors. A lattice L spanned by $b_1, \dots, b_d$ is the set of linear combinations of $b_1, \dots, b_d$ with integer coefficients, and the set is called a basis of L . Any two bases B_1 and B_2 of the same lattice L are related by a square matrix U of integer coordinates. The determinant of U equals ± 1 and $B_2 = UB_1$.

There is a fundamental lattice problem known as the shortest vector problem (SVP), which is NP-hard under randomized reductions[1]. SVP asks for a nonzero vector that minimizes the Euclidean norm of the vectors in a lattice L . The Gaussian heuristic suggests that the length of the shortest vector is approximately equal to

$$\sqrt{2\pi e} \det(L)^{1/d}.$$

The LLL algorithm[8] is a lattice reduction algorithm often used to compute a short vector in a lattice. Given a basis $b_1, \dots, b_d$, the algorithm finds a vector b'_1 in polynomial time and guarantees that

$$\|b'_1\| \leq 2^{(d-1)/4} \det(L)^{1/d}.$$

The BKZ algorithm[10] is a variant of the LLL algorithm. Given a basis $b_1, \dots, b_d$ and an integer parameter $\beta > 1$, the algorithm finds a vector b'_1 and guarantees that

$$\|b'_1\| \leq \left(\frac{\beta}{\pi e}\right)^{(n-1)/(\beta-1)} \min_{v \in L, v \neq 0} \|v\|.$$

However, the BKZ is no longer a polynomial-time algorithm.

3.2 Coppersmith's method

One important application found by Coppersmith of lattice reduction algorithm is finding small roots of modular equations[2, 6]. Let d be an integer of unknown factorization and $p(x)$ be a monic polynomial of degree N with integer coefficients. The modular equation $p(x) \equiv 0 \pmod{d}$ is an instance of the problem to solve.

Theorem 1 (Coppersmith). Given a monic polynomial $p(x)$ of degree N , one can find all x_0 such

$$\begin{pmatrix}
a_0^3 & 3a_0^2X & 3a_0X^2 & X^3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & a_0^2X & 3a_0X^2 & 3a_0X^3 & X^4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & a_0^3X^2 & 3a_0^2X^3 & 3a_0X^4 & X^5 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
a_0^2b_0 & 2a_0b_0X & b_0X^2 & 0 & 0 & 0 & a_0^2Y & 2a_0XY & X^2Y & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & a_0^2b_0X & 2a_0b_0X^2 & b_0X^3 & 0 & 0 & 0 & a_0^2XY & 2a_0X^2Y & X^3Y & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & a_0^2b_0X^2 & 2a_0b_0X^3 & b_0X^4 & 0 & 0 & 0 & a_0^2X^2Y & 2a_0X^3Y & X^4Y & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
a_0b_0^2 & b_0^2X & 0 & 0 & 0 & 0 & 2a_0b_0Y & 2b_0XY & 0 & 0 & 0 & a_0Y^2 & XY^2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & a_0b_0^2X & b_0^2X^2 & 0 & 0 & 0 & 0 & 2a_0b_0XY & 2b_0X^2Y & 0 & 0 & 0 & a_0XY^2 & X^2Y^2 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & a_0b_0^2X^2 & b_0^2X^3 & 0 & 0 & 0 & 0 & 2a_0b_0X^2Y & 2b_0X^3Y & 0 & 0 & 0 & a_0X^2Y^2 & X^3Y^2 & 0 & 0 & 0 & 0 & 0 \\
b_0^3 & 0 & 0 & 0 & 0 & 0 & 3b_0^2Y & 0 & 0 & 0 & 0 & 3b_0Y^2 & 0 & 0 & 0 & Y^3 & 0 & 0 & 0 & 0 \\
0 & b_0^3X & 0 & 0 & 0 & 0 & 0 & 3b_0^2XY & 0 & 0 & 0 & 0 & 3b_0XY^2 & 0 & 0 & 0 & XY^3 & 0 & 0 & 0 \\
0 & 0 & b_0^3X^2 & 0 & 0 & 0 & 0 & 0 & 3b_0^2X^2Y & 0 & 0 & 0 & 0 & 3b_0X^2Y^2 & 0 & 0 & 0 & X^2Y^3 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & b_0^3Y & 0 & 0 & 0 & 0 & 3b_0^2Y^2 & 0 & 0 & 0 & 3b_0Y^3 & 0 & 0 & Y^4 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & b_0^3XY & 0 & 0 & 0 & 0 & 3b_0^2XY^2 & 0 & 0 & 0 & 3b_0XY^3 & 0 & 0 & XY^4 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & b_0^3Y^2 & 0 & 0 & 0 & 3b_0^2Y^3 & 0 & 0 & 3b_0Y^4 & Y^5
\end{pmatrix}$$

Figure 1: The lattice basis for $h = 5$ and $u = 3$

that $p(x) \equiv 0 \pmod{d}$ and $|x_0| < cd^{1/N}$ where c is a constant.

We choose integers $h > u \geq 1$ and construct a set of polynomials $p_1(x), p_2(x), \dots, p_h(x)$ such that they all have the same small roots x_0 modulo d^u . For example, take the polynomials

$$\begin{aligned}
p_i(x) &= d^{u-i}p^i(x) \quad \text{for } i = 1, \dots, u, \\
p_i(x) &= x^{i-u}p^u(x) \quad \text{for } i = u+1, \dots, h.
\end{aligned}$$

If $r(x)$ is a linear combination of these polynomials then $r(x)$ also has the property $r(x_0) \equiv 0 \pmod{d^u}$. The trick is to find a small $r(x)$ such that $|r(x)| < d^u$ for all $|x| < X$. Therefore, we only need to solve $r(x) = 0$ in real numbers instead of modular arithmetic.

To find such an $r(x)$, one needs to take advantage of the lattice reduction algorithm. Each polynomial $p_i(x) = a_nx^n + a_{n-1}x^{n-1} + \dots + a_1x + a_0$ is associated with a row vector

$$v_i = (a_0, a_1X, \dots, a_{n-1}X^{n-1}, a_nX^n).$$

After reducing the lattice basis $\{v_1, v_2, \dots, v_h\}$ by applying the LLL algorithm, we are guaranteed that if $X < cd^{1/N}$, then the polynomial $r(x)$ associated with v'_1 is the desired polynomial.

Coppersmith's method can be naturally extended to modular bivariate polynomials $p(x, y)$. The idea is to find two algebraically independent polynomials $r_1(x, y)$ and $r_2(x, y)$. Then we can solve them for all roots. The problem is that lattice reduction algorithm does not guarantee the independence of two small vectors, so this is a heuristic algorithm.

3.3 Newton's method

Newton's method is applicable to solve a system of two algebraically independent polynomials

$r_1(x, y)$ and $r_2(x, y)$, which are continuously differentiable. The system is a function $F : R^2 \rightarrow R^2$ as follows:

$$F \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} r_1(x, y) \\ r_2(x, y) \end{pmatrix}.$$

The Jacobian matrix J_F is

$$J_F \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \frac{\partial}{\partial x}r_1(x, y) & \frac{\partial}{\partial y}r_1(x, y) \\ \frac{\partial}{\partial x}r_2(x, y) & \frac{\partial}{\partial y}r_2(x, y) \end{pmatrix},$$

and the relation of Newton's method is

$$J_F \begin{pmatrix} x_n \\ y_n \end{pmatrix} \left(\begin{pmatrix} x_{n+1} \\ y_{n+1} \end{pmatrix} - \begin{pmatrix} x_n \\ y_n \end{pmatrix} \right) = -F \begin{pmatrix} x_n \\ y_n \end{pmatrix}.$$

If the initial guess (x_1, y_1) is reasonably close to the solution (x_0, y_0) , the sequence of iterates converges to the solution.

3.4 Algorithm for ACDP

The lattice solution to the ACDP comes from Coppersmith's method. The inputs can be transformed into two bivariate polynomials $q_1(x, y) = a_0 + x$ and $q_2(x, y) = b_0 + y$. Then we have $q_1(x_0, y_0) \equiv 0 \pmod{d}$ and $q_2(x_0, y_0) \equiv 0 \pmod{d}$ for some $x_0 < X$ and $y_0 < Y$. It is obvious that we can solve the modular equations by applying Coppersmith's Method.

Though the modulus d is not known, the required polynomials can be produced by setting $p_i(x, y) = q_1(x, y)^{u-i}q_2(x, y)^i$ for $i = 0, \dots, u$ where $u \geq 1$. Then we can guarantee that $p_i(x_0, y_0) \equiv 0 \pmod{d^u}$. Next, an integer h is chosen to be the bound on the total degree of all polynomials. For example, with $h = 5$ and $u = 3$, Figure 1 shows the built lattice basis.

Theorem 2 (Howgrave-Graham). Given a_0, b_0, X , and D satisfying following conditions:

- a_0 and b_0 have the same bit-length.
- $a_0 < b_0$.
- $D = b_0^{\alpha_0}$ and $X = b_0^{\beta_0}$, where $\alpha_0 \in (0, 2/3)$, and $\beta_0 = 1 - (1/2)\alpha_0 - \sqrt{1 - \alpha_0 - (1/2)\alpha_0^2} - \epsilon$.

Then there exists a polynomial-time algorithm which outputs all d 's such that $d \mid a_0 + x_0$ and $d \mid b_0 + y_0$ for some $d > D$, $|x_0| < X$ and $|y_0| < X$.

For the inputs satisfying the conditions above, the LLL lattice reduction algorithm is performed on the constructed lattice basis to find out the appropriate vectors r_1 and r_2 .

Finally, bivariate polynomials $r_1(x, y)$ and $r_2(x, y)$ can be derived from r_1 and r_2 . Since $r_1(x_0, y_0) = 0$ and $r_2(x_0, y_0) = 0$, we can apply Newton's method on the bivariate polynomials to find the integer solutions x_0 and y_0 . The solution to the ACDP is therefore the greatest common divisor of $a + x_0$ and $b + y_0$.

4 Computational Experiments

Recall the example of fully homomorphic encryption in Section 2. Suppose only the integers $\{x_0 = 1020807, x_1 = 760542, x_2 = 955735, x_3 = 854075\}$ of the public key obtained. We choose x_0 and x_1 to perform the algorithm.

Example 2. Given the input $a_0 = 760542$, $b_0 = 1020807$, $D = 4000$, and $X = 14$, the algorithm first computes $\alpha_0 = 0.599$, and $\beta_0 = 0.191$, and checks the condition in Theorem 2. The algorithm is described below step by step.

1. Construct two bivariate polynomials:
 $q_1(x, y) = a_0 + x = 760542 + x$, and
 $q_2(x, y) = b_0 + y = 1020807 + y$.
2. Choose integers h and u with $h > u \geq 1$. Here $h = 7$ and $u = 5$.
3. Construct $u + 1$ bivariate polynomials $p_i(x, y) = q_1(x, y)^{u-i} q_2(x, y)^i$ for $i = 0, \dots, u$.
 So, $p_0(x, y) = x^5 + 3802710x^4 + 5784241337640x^3 + 4399158475411400880x^2 + 1672872392603168824038480x + 254457943043039844754374731232$,
 $p_1(x, y) = x^4y + 1020807x^4 + 3042168x^3y + 3105466389576x^3 + 3470544802584x^2y + 3542756428291365288x^2 + 1759663390164560352xy + 1796276706323714359244064x + 334574478520633764807696y$

$$\begin{aligned}
 &+ 341535969695212591552049730672, \\
 p_2(x, y) &= x^3y^2 + 2041614x^3y \\
 &+ 1042046931249x^3 + 2281626x^2y^2 \\
 &+ 4658199584364x^2y \\
 &+ 2377561371557930874x^2 \\
 &+ 1735272401292xy^2 \\
 &+ 3542756428291365288xy \\
 &+ 1808235280647411862773708x \\
 &+ 439915847541140088y^2 \\
 &+ 898138353161857179622032y \\
 &+ 458412958938047970979213809912, \\
 p_3(x, y) &= x^2y^3 + 3062421x^2y^2 \\
 &+ 3126140793747x^2y \\
 &+ 1063728801747497943x^2 \\
 &+ 1521084xy^3 + 4658199584364xy^2 \\
 &+ 4755122743115861748xy \\
 &+ 1618020860677291161130212x \\
 &+ 578424133764y^3 \\
 &+ 1771378214145682644y^2 \\
 &+ 1808235280647411862773708y \\
 &+ 615286410710614187134146847452, \\
 p_4(x, y) &= xy^4 + 4083228xy^3 \\
 &+ 6252281587494xy^2 \\
 &+ 4254915206989991772xy \\
 &+ 1085861806925458132700001x \\
 &+ 760542y^4 + 3105466389576y^3 \\
 &+ 4755122743115861748y^2 \\
 &+ 3236041721354582322260424y \\
 &+ 825843510362701779159924160542, \\
 p_5(x, y) &= y^5 + 5104035y^4 + 10420469312490y^3 \\
 &+ 10637288017474979430y^2 \\
 &+ 5429309034627290663500005y \\
 &+ 1108455333542156140067089920807.
 \end{aligned}$$

4. Construct a lattice basis, in which each vector is associated with a polynomial as mentioned in Coppersmith's method. The polynomials are of the form $x^m y^n p_i(x, y)$ such that $m + n \leq h - u$. In this case, the lattice basis contains 21 vectors of 36 dimensions. The description is omitted.
5. By applying LLL algorithm to the lattice basis constructed in previous step to find the small vectors r_1 and r_2 , where $r_1 = (-69311993709514752, -68057883165049344, -203295069521159616, 102360742741810728, 7684347031919712, -8681573771603712, 967837665057984, 0, 12479978050864128, 131053622441930496, -45164652904079856, -90452204055353952, 43173041632576128, -4973027589601344, 0, 15203142773602560, -103167081841289760, 176586299912586816, -80474343512187744, 10467194017851648, 0, 59601473473768128, -125213598363412608,$

71985207681740864, -11594585931511296, 0, 30308545175930112, -31297338469496960, 7147549627681792, 0, 5331396153475840, -2329400023172096, 0, 314002131537920, 0, 0), and $r_2 = (-92630193283031040, 247826997950063616, -121573848035332224, -119048296351528656, 96688423819853712, -22790381749380000, 1769874450446592, 0, 110577934440142848, -1765603295027712, 104733196878685536, -151533474055218864, 52415972378417088, -5382477033405696, 0, 68819451977074176, 42174736289856576, 16667347621113504, -29698343248057344, 5313907710030528, 0, -40324008826394496, 67560064892031936, -13940478461023744, -597987730164160, 0, -26710052373341568, 18629799012561408, -2281628174243712, 0, -4778318427911168, 1459577151802112, 0, -278057150690816, 0, 0).$

6. Eliminate the factors of X and Y in the elements of r_1 and r_2 , the bivariate polynomials $r_1(x, y)$ and $r_2(x, y)$ are derived as follows:
 $r_1(x, y) = 128538819x^6 - 660469329x^5y - 16142034888x^5 + 1390151268x^4y^2 + 80273549772x^4y + 200029858182x^4 - 1539880536x^3y^3 - 149629513581x^3y^2 - 2354545086822x^3y + 37303477675587x^3 + 949268272x^2y^4 + 133845287086x^2y^3 + 4596686274276x^2y^2 - 16459421612274x^2y - 1037219742454896x^2 - 309368336xy^5 - 58192528540xy^4 - 3259412702088xy^3 - 37597333032540xy^2 + 668640930826176xy - 4861277368932096x + 41702720y^6 + 9912901160y^5 + 788956298832y^4 + 21720653598312y^3 + 77567054967360y^2 + 891427003633152y - 69311993709514752.$
 $r_2(x, y) = 235057572x^6 - 714848436x^5y - 42375166875x^5 + 705741723x^4y^2 + 97459340562x^4y + 2516879004057x^4 - 79418935x^3y^3 - 55219445856x^3y^2 - 3944540661579x^3y - 43384947649974x^3 - 303023742x^2y^4 - 25920149456x^2y^3 + 433864733994x^2y^2 + 38168074664244x^2y - 620274734874144x^2 + 193846892xy^5 + 34639210992xy^4 + 1758643921596xy^3 + 15369801854904xy^2 - 9008180076672xy + 17701928425004544x - 36928856y^6 - 8884539232y^5 - 695284578648y^4 - 14695338493584y^3 + 351119652944256y^2 + 7898423888581632y - 92630193283031040.$
7. Newton's method is applied to solve the equations. The initial guess is $(-7, 5)$, and the iterative answers are $(-8.2, 20.2)$,

$(-8.4, 14.4), (-17.7, 7.7), (-14.4, 9.6), (-13.2, 10.0), (-13.0, 10.0).$

8. If the solution (x_0, y_0) is integral derived from Newton's method, the greatest approximate common divisor is computed by $\gcd(a_0 + x_0, b_0 + y_0) = \gcd(760529, 1020817) = 4067$, the private key set in Example 1.

The algorithm works for Example 2 if $u = 4$ and $h = 6$ are chosen. But it doesn't work if $u = 3$ and $h = 5$ since the higher error bound requires larger u and h . This causes a worse performance in lattice basis reduction. Therefore, we use a tricky divide-and-conquer strategy to prevent the situation.

4.1 Divide-and-conquer strategy

In Example 2, u and h should be large enough, or the algorithm might fail. Since the error bound is known, the exact multiples of the common divisor d lie in a square region. It is a simple observation that the square can be partitioned into some small squares, as shown in Figure 2. This strategy reduces the original error bound X to $X/3$ in Figure 2. In this way, the parameter u and h can be still small for efficiency. Furthermore, it helps for some case the error bound X is too big to satisfy the conditions in Theorem 2.

Again, Example 2 is reviewed by using the divide-and-conquer strategy. The original square is partitioned into 4 small squares. This becomes 4 easier cases as follows:

1. $a_0 = 760549, b_0 = 1020814, D = 4000$, and $X = 7$, the algorithm first computes $\alpha_0 = 0.599$, and $\beta_0 = 0.141$.
2. $a_0 = 760549, b_0 = 1020800, D = 4000$, and $X = 7$, the algorithm first computes $\alpha_0 = 0.599$, and $\beta_0 = 0.141$.
3. $a_0 = 760535, b_0 = 1020814, D = 4000$, and $X = 7$, the algorithm first computes $\alpha_0 = 0.599$, and $\beta_0 = 0.141$.

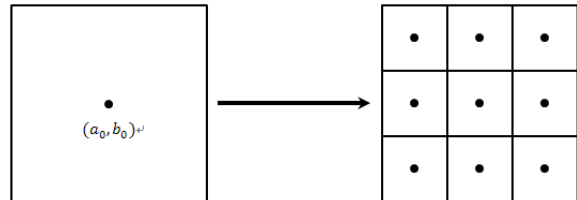


Figure 2: The divide-and-conquer squares

$$\begin{pmatrix} 578413486225 & 1521070 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 578413486225 & 1521070 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 578413486225 & 1521070 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 776364775490 & 1020814 & 0 & 0 & 0 & 760535 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 776364775490 & 1020814 & 0 & 0 & 0 & 760535 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 776364775490 & 1020814 & 0 & 0 & 760535 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1042061222596 & 0 & 0 & 0 & 0 & 2041628 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1042061222596 & 0 & 0 & 0 & 0 & 2041628 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1042061222596 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1042061222596 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1042061222596 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1042061222596 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1042061222596 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1042061222596 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Figure 3: The lattice basis for $h = 4$ and $u = 2$

4. $a_0 = 760535$, $b_0 = 1020800$, $D = 4000$, and $X = 7$, the algorithm first computes $\alpha_0 = 0.599$, and $\beta_0 = 0.141$.

For all cases, $u = 3$ and $h = 5$ is set, and the initial guess is $(0, 0)$. In case 1, the solution is $(3.17, 3.17)$. In case 2, that is $(1.06, -3.38)$. And in case 4, that is $(14.63, -23.75)$. They are not integral. But in case 3, the algorithm works even when $u = 2$ and $h = 4$. The following shows the steps of the algorithm for case 3.

1. Construct two bivariate polynomials:
 $q_1(x, y) = a_0 + x = 760535 + x$, and
 $q_2(x, y) = b_0 + y = 1020814 + y$.
2. $h = 4$, $u = 2$.
3. Construct $u + 1$ bivariate polynomials
 $p_i(x, y) = q_1(x, y)^{u-i} q_2(x, y)^i$ for $i = 0, \dots, u$.
 So, $p_0(x, y) = x^2 + 1521070x + 578413486225$,
 $p_1(x, y) = xy + 1020814x + 760535 * y + 776364775490$, and
 $p_2(x, y) = y^2 + 2041628y + 1042061222596$.
4. Construct a lattice basis shown in Figure 3.
5. Apply LLL algorithm to the lattice basis constructed in previous step to find the small vectors r_1 and r_2 , where $r_1 = (-1119252, 296478, -407386, -637294, 1234114, 348166, -678552, 1532524, 485002, 1110977, 856471, -1423793, -1225196, -727503, 751513)$, and $r_2 = (-159003, -93177, -764204, -960743, -559433, 1508976, -246176, -730933, -655473, -2365916, 789586, 1445402, 214718, -136857, -256907)$.
6. Find the bivariate polynomials $r_1(x, y)$ and $r_2(x, y)$ where
 $r_1(x, y) = 514x^4 + 202x^3y - 1858x^3 - 593x^2y^2 + 4468x^2y - 8314x^2 - 303xy^3 + 2497xy^2 - 13848xy + 42354x + 313y^4 - 3572y^3 + 22673y^2 + 49738y - 1119252$, and $r_2(x, y) = -233x^4 - 273x^3y - 2801x^3 + 602x^2y^2 - 2131x^2y - 15596x^2 - 57xy^3 + 2302xy^2 - 5024xy - 13311x - 107y^4 + 626y^3 - 48284y^2 + 215568y - 159003$.

7. The iteration answers starting with the initial guess are $(0, 0)$, $(23.8, 2.2)$, $(24.8, -43.2)$, $(102., -31.8)$, $(15.2, -23.4)$, $(12.7, -17.2)$, $(11.4, -12.8)$, $(11.6, -9.99)$, $(17.9, -8.7)$, $(15.0, -9.2)$, $(12.3, -9.2)$, $(4.2, -9.0)$, $(9.9, -7.3)$, $(-0.7, -6.4)$, $(5.3, -3.2)$, $(-2.2, -5.0)$, $(3, 7, -1.9)$, $(-8.8, -7.5)$, $(-8.8, -2.5)$, $(-7.0, 0.2)$, $(-6.4, 1.6)$, $(-6.2, 2.3)$, $(-6.1, 2.7)$, $(-6.0, 2.9)$, $(-6.0, 2.9)$, $(-6.0, 3.0)$, ..., $(-6, 3)$.

8. $d = \gcd(760535 - 6, 1020814 + 3) = 4067$.

This example shows that the parameters u and h really decrease by use of the divide-and-conquer strategy. Also, the small cases can be run in parallel to speed up the algorithm.

5 Conclusion

In this paper, we propose a strategy applied to the lattice approach for solving the approximate common divisor problem. While we partition the search region into small squares, the computation for each small square is twice faster. Furthermore, by using the strategy, the original algorithm might handle more cases. Finally, we can speed up this algorithm by using parallel computers.

References

- [1] M. Ajtai, "The shortest vector problem in L_2 is NP-hard for randomized reductions," *In Proc. 30th ACM Symposium on Theory of Computing*, 1998.
- [2] D. Coopersmith. "Finding a small root of a univariate Modular Equation," *In Proc. of Eurocrypt '96*, Vol. 1070, Lecture Notes in Computer Science, 1996.
- [3] D. Coopersmith. "Finding a small root of a bivariate integer equation," *In Proc. of Eurocrypt '96*, Vol. 1070, Lecture Notes in Computer Science, 1996

- [4] Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan, “Fully homomorphic encryption over the integers,” *Advances in Cryptology – EUROCRYPT 2010*.
- [5] Craig Gentry, “Fully homomorphic encryption using ideal lattices,” *In Proc. 41st annual ACM symposium on Theory of computing (STOC '09)*, 2009.
- [6] N.A. Howgrave-Graham, “Finding small roots of univariate modular equations revisited,” *In Cryptography and Coding*, Vol. 1355, Lecture Notes in Computer Science, 1997.
- [7] N.A. Howgrave-Graham, “Approximate integer common divisors,” *In Proc. of CALC 2001*, LNCS, 2001.
- [8] A.K. Lenstra, H.W. Lenstra, Jr., and L. Lovasz, “Factoring polynomials with rational coefficients,” *Mathematische Ann.* 261, pp. 513534, 1982.
- [9] Ronald L. Rivest, Len Adleman, and Michael L. Dertouzos, “On data banks and privacy homomorphisms,” *Foundations of Secure Computation*, 1978.
- [10] C.-P. Schnorr and M. Euchner, “Lattice basis reduction: improved practical algorithms and solving subset sum problems,” *Math. Programming*, Vol. 66, pp. 181199, 1994.